

Tugas 5: Praktikum Mandiri 5 Machine Learning

Hisyam Wildan Alfath - 0110222206

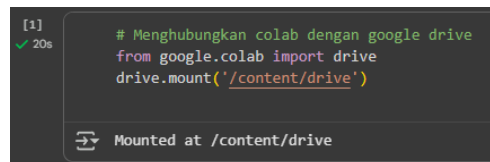
Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hisy22206ti@student.nurulfikri.ac.id

Abstract. Praktikum ini membahas implementasi Decision Tree untuk klasifikasi pada dataset Iris menggunakan Python. Proses yang dilakukan meliputi praproses data, encoding label, eksplorasi fitur, pemisahan data, pelatihan model, evaluasi performa, tuning hyperparameter, dan visualisasi hasil model. Model yang dihasilkan memperoleh akurasi tinggi dan interpretasi yang kuat dari visualisasi pohon keputusan. Fitur pada petal terbukti paling berpengaruh dalam membedakan spesies iris. Keseluruhan hasil menunjukkan bahwa Decision Tree sangat cocok digunakan untuk masalah klasifikasi sederhana dengan data terstruktur seperti dataset Iris.

1. Praktikum Mandiri 05

- Menghubungkan Colab dengan Google Drive

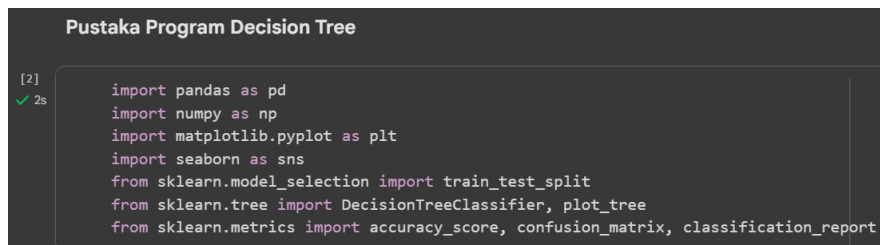


```
[1]
✓ 20s # Menghubungkan colab dengan google drive
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive
```

Kode ini memanggil modul drive dari google.colab untuk menghubungkan (mount) Google Drive ke lingkungan Google Colab, dan outputnya adalah muncul pesan “Mounted at /content/drive” yang menandakan folder Google Drive sekarang sudah bisa diakses pada path tersebut di Colab.

- Import Library Machine Learning



```
Pustaka Program Decision Tree

[2]
✓ 2s import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Kode ini berfungsi untuk mengimpor pustaka seperti pandas, numpy, matplotlib, seaborn, dan modul dari scikit-learn yang dibutuhkan dalam membangun serta mengevaluasi model machine learning di Python. Dengan langkah ini, berbagai

fungsi untuk analisis data dan pemodelan pohon keputusan menjadi tersedia dan siap digunakan dalam proses selanjutnya.

- Loading Dataset

```
Loading Dataset

[3] ✓ ds
df = pd.read_csv("/content/drive/MyDrive/Hisyam Wildan Alfath/Semester 7/Machine Learning/Pertemuan 5/Praktikum & Tugas Mandiri 05/data/Iris.csv")
df.head()
```

	Id	SepallengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

Kode pada bagian ini digunakan untuk membaca data dari file CSV dan menampilkannya dalam bentuk DataFrame menggunakan fungsi `pd.read_csv()`. Dengan langkah ini, data mentah yang berisi fitur dan label dapat diakses dan divisualisasikan sehingga siap untuk diproses pada tahapan machine learning selanjutnya.

- Data Preprocessing

```
Data Preprocessing

[4] ✓ Os
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   Id              150 non-null   int64  
 1   SepalLengthCm   150 non-null   float64 
 2   SepalWidthCm    150 non-null   float64 
 3   PetalLengthCm   150 non-null   float64 
 4   PetalWidthCm    150 non-null   float64 
 5   Species         150 non-null   object  
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

Kode pada bagian data preprocessing menggunakan fungsi `df.info()` untuk menampilkan detail ringkasan isi DataFrame, yaitu jumlah baris total (150), nama-nama kolom, jumlah data yang tidak kosong di setiap kolom, serta tipe data di masing-masing kolom seperti float64, int64, dan object; dari output ini dapat diketahui bahwa semua data lengkap tanpa missing value dan tipe data sudah sesuai untuk analisis lebih lanjut.

- Cek Missing Value

```
[5]
✓ 0s # Cek missing value
      df.isna().sum()

      0
      Id      0
      SepalLengthCm  0
      SepalWidthCm  0
      PetalLengthCm  0
      PetalWidthCm  0
      Species      0

      dtype: int64
```

Kode ini digunakan untuk memeriksa apakah terdapat data yang kosong (missing value) pada setiap kolom dengan fungsi `df.isna().sum()`. Output yang dihasilkan menunjukkan nilai nol pada semua kolom, artinya seluruh data sudah lengkap dan tidak ada nilai yang hilang, sehingga dapat langsung digunakan untuk proses analisis selanjutnya.

- Rename Kolom

```
[7]
      df = df.rename(columns={
          'SepalLengthCm': 'Sepal Length (cm)',
          'SepalWidthCm': 'Sepal Width (cm)',
          'PetalLengthCm': 'Petal Length (cm)',
          'PetalWidthCm': 'Petal Width (cm)'
      })
```

Kode ini digunakan untuk mengganti nama kolom pada DataFrame agar menjadi lebih mudah dibaca, misalnya dari `SepalLengthCm` menjadi `Sepal Length (cm)`. Output dari langkah ini adalah DataFrame dengan nama-nama kolom baru yang lebih jelas, sehingga analisis data berikutnya menjadi lebih nyaman dan informatif.

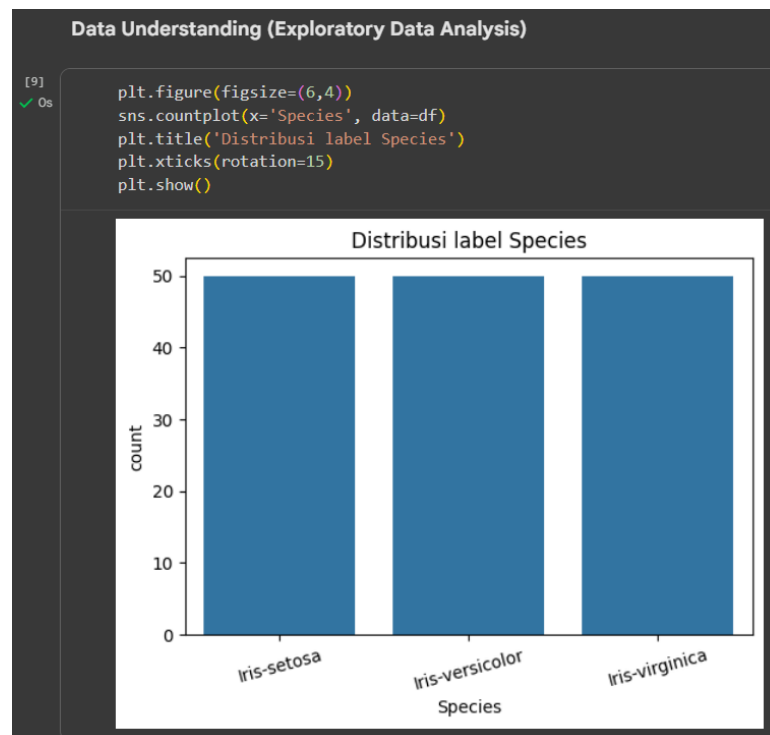
- Cek Perubahan Nama Kolom

```
[8]
✓ Os df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Id                   150 non-null    int64
1   Sepal Length (cm)   150 non-null    float64
2   Sepal Width (cm)    150 non-null    float64
3   Petal Length (cm)   150 non-null    float64
4   Petal Width (cm)    150 non-null    float64
5   Species              150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

Kode ini menjalankan kembali fungsi `df.info()` setelah proses perubahan nama kolom sebelumnya. Output menampilkan DataFrame dengan nama kolom baru seperti "Sepal Length (cm)" dan "Petal Width (cm)" yang lebih deskriptif, serta memastikan jumlah data dan tipe datanya masih tetap lengkap dan sesuai.

- Data Understanding (Exploratory Data Analysis)



Kode ini digunakan untuk membuat visualisasi distribusi jumlah data pada masing-masing label spesies bunga iris menggunakan countplot dari seaborn. Output yang dihasilkan adalah grafik batang yang menunjukkan setiap kelas spesies (Iris-setosa, Iris-versicolor, Iris-virginica) memiliki jumlah data yang seimbang, masing-masing sekitar 50 sampel, sehingga data terdistribusi merata untuk ketiga kelas.

- Encoding Data Kategorikal (Mapping Label ke Kode Numerik)

```

Encoding Data Kategorikal (Mapping Label ke Kode Numerik)

[10]
✓ Os
# mapping label -> kode untuk target
species_cat = df['Species'].astype('category')
species_classes = list(species_cat.cat.categories) # urutan kelas
df['Species'] = species_cat.cat.codes # y numerik

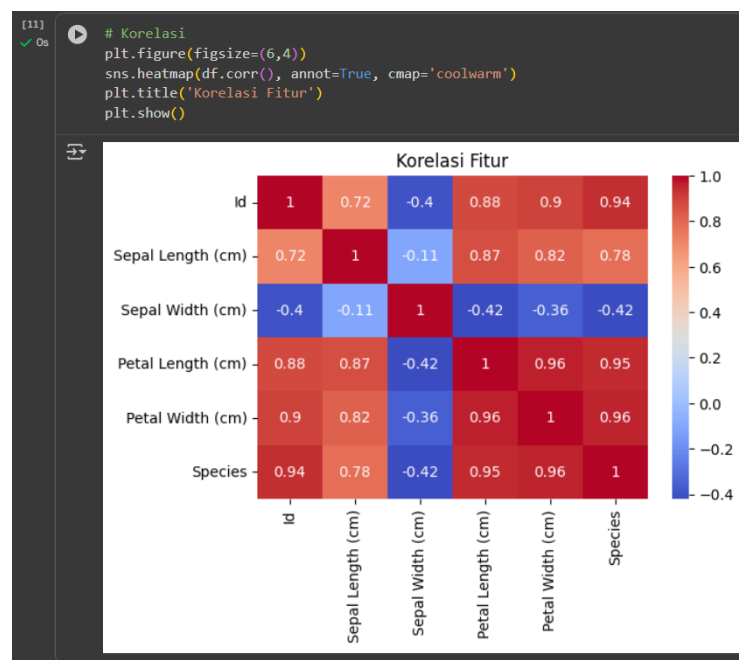
df.head()

```

	Id	Sepal Length (cm)	Sepal Width (cm)	Petal Length (cm)	Petal Width (cm)	Species
0	1	5.1	3.5	1.4	0.2	0
1	2	4.9	3.0	1.4	0.2	0
2	3	4.7	3.2	1.3	0.2	0
3	4	4.6	3.1	1.5	0.2	0
4	5	5.0	3.6	1.4	0.2	0

Kode ini digunakan untuk mengubah nilai label kategorikal pada kolom "Species" menjadi kode numerik menggunakan metode kategori pandas. Output yang dihasilkan adalah DataFrame dengan kolom "Species" yang sebelumnya berisi nama spesies sekarang sudah diubah menjadi angka (misal 0 untuk Iris-setosa), sehingga data lebih siap digunakan dalam pemodelan machine learning berbasis numerik.

- Korelasi Fitur



Kode ini digunakan untuk menampilkan matriks korelasi antar fitur pada dataset dalam bentuk heatmap menggunakan seaborn. Output berupa grafik korelasi yang menunjukkan tingkat hubungan antar fitur, di mana angka dan warna pada matriks memudahkan melihat fitur mana saja yang saling terkait kuat, serta membantu dalam memilih fitur yang relevan untuk analisis model machine learning selanjutnya.

- Splitting Data (Pembagian Data Training dan Testing)

```
[12]
✓ Os
# Memilih fitur dan target
feature_cols = ['Sepal Length (cm)', 'Sepal Width (cm)', 'Petal Length (cm)', 'Petal Width (cm)']
X = df[feature_cols]
y = df['Species']
```

Kode ini digunakan untuk memilih fitur (variabel input) dan target (label output) dari DataFrame sebelum dilakukan pembagian data. Output dari langkah ini adalah dua variabel, yaitu X yang berisi data fitur seperti panjang dan lebar sepal/petal, serta y yang berisi data target berupa label spesies, sebagai persiapan untuk proses training dan testing pada model machine learning.

- Membagi Dataset

```
[13]
✓ Os
# Membagi dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
len(X_train), len(X_test)

(120, 30)
```

Kode ini digunakan untuk membagi data menjadi data training dan testing menggunakan fungsi `train_test_split`, dengan rasio 80% data untuk pelatihan (120 data) dan 20% untuk pengujian (30 data). Output dari proses ini memastikan bahwa model machine learning nantinya dilatih dan diuji dengan data yang terpisah, sehingga performanya bisa dievaluasi secara objektif.

- Pembuatan Model Decision Tree

```
Pembuatan Model Decision Tree

[14]
✓ Os
# Membangun model
dt = DecisionTreeClassifier(
    criterion='gini',
    max_depth=4,
    random_state=42
)
dt.fit(X_train, y_train)

DecisionTreeClassifier
DecisionTreeClassifier(max_depth=4, random_state=42)
```

Kode ini digunakan untuk membangun model pohon keputusan (Decision Tree) menggunakan `DecisionTreeClassifier` dengan kriteria 'gini' dan kedalaman maksimum

4, lalu melatihnya menggunakan data training yang sudah dipisahkan sebelumnya. Output dari proses ini adalah objek model yang telah terlatih serta siap digunakan untuk melakukan prediksi terhadap data uji dan proses evaluasi selanjutnya.

- Evaluasi Model Decision Tree

```
Evaluasi Model Decision Tree

[15]
✓ Os
# Evaluasi
y_pred = dt.predict(X_test)

print("Akurasi:", round(accuracy_score(y_test, y_pred)*100, 2), "%")
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(
    y_test, y_pred, target_names=species_classes
))

Akurasi: 93.33 %

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classification Report:

```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	0.90	0.90	0.90	10
Iris-virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Kode ini digunakan untuk mengevaluasi model decision tree dengan melakukan prediksi pada data uji, lalu menghitung akurasi, menampilkan confusion matrix, serta classification report. Output dari langkah ini menunjukkan tingkat akurasi model sebesar 93.33%, perincian prediksi benar dan salah antar kelas dalam confusion matrix, serta metrik evaluasi seperti precision, recall, dan f1-score pada classification report, sehingga kualitas model dapat dianalisis dengan lengkap.

- Visualisasi Hasil Model Decision Tree

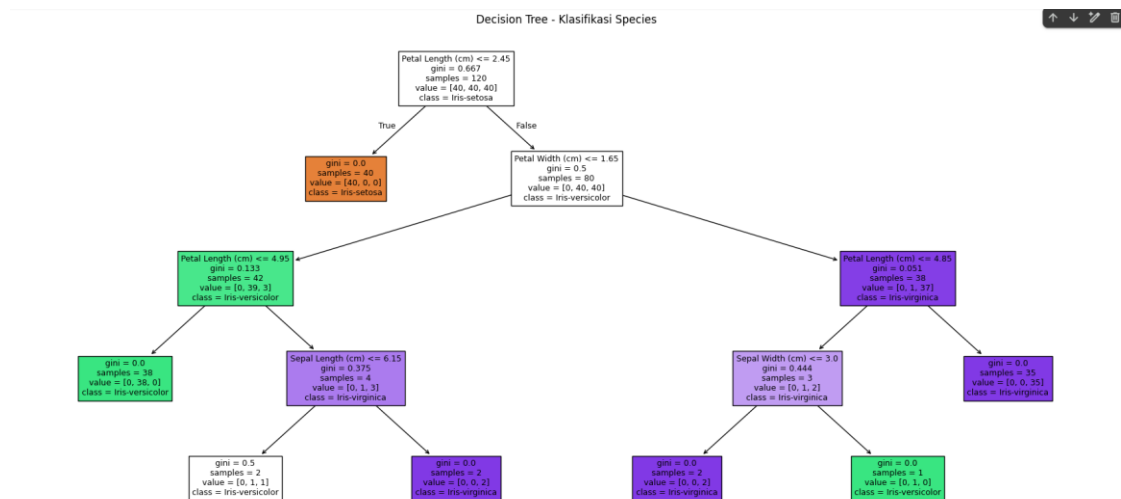
```
Visualisasi Hasil Model Decision Tree

[16]
✓ Os
# Visualisasi model
plt.figure(figsize=(22,10))
plot_tree(
    dt,
    feature_names=feature_cols,
    class_names=species_classes, # kembali ke nama kelas asli
    filled=True,
    fontsize=9
)
plt.title("Decision Tree - Klasifikasi Species")
plt.show()
```

Kode ini digunakan untuk menampilkan visualisasi pohon keputusan hasil pelatihan model dengan `plot_tree` dari `sklearn`. Output dari langkah ini adalah gambar

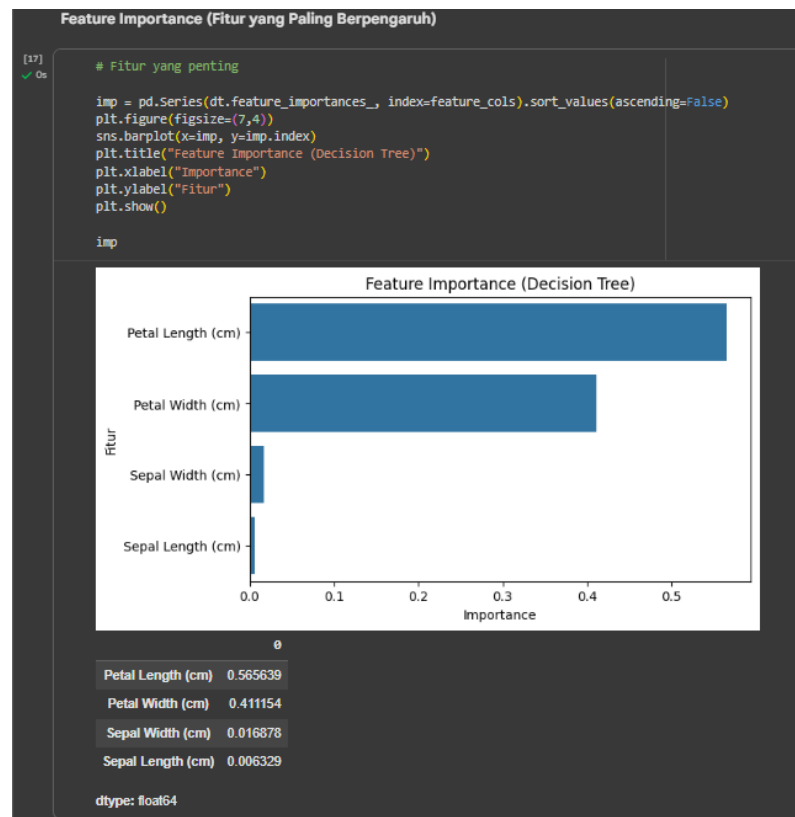
diagram decision tree yang memperlihatkan bagaimana fitur dan cabang pohon membagi data menjadi kelas-kelas spesies, sehingga proses pengambilan keputusan oleh model bisa dipahami secara visual dan lebih mudah dianalisis

- Output Visualisasi Model Decision Tree



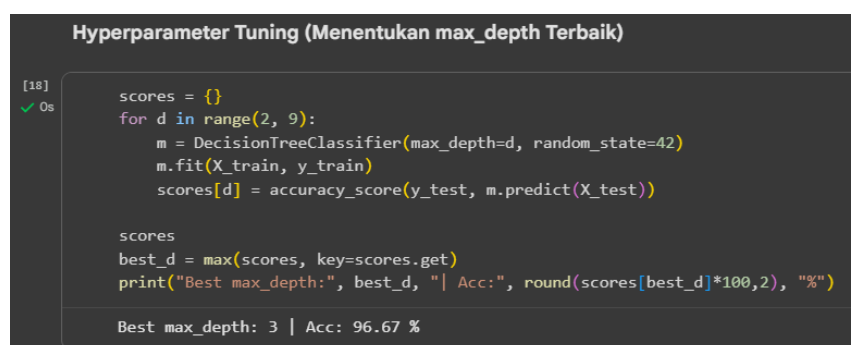
Output dari kode sebelumnya menghasilkan gambar diagram pohon keputusan yang memperlihatkan bagaimana model melakukan klasifikasi berdasarkan fitur bunga, seperti panjang dan lebar sepal serta petal. Setiap cabang pada pohon menunjukkan proses pemilihan atribut yang memisahkan data ke kelas spesies yang tepat (Iris-setosa, Iris-versicolor, Iris-virginica), sehingga konsep pengambilan keputusan model dapat dipahami secara visual dan logis.

- Feature Importance (Fitur yang Paling Berpengaruh)



Kode ini digunakan untuk mengetahui fitur mana yang paling berpengaruh dalam proses klasifikasi menggunakan model decision tree, dengan memvisualisasikan nilai "importance" tiap fitur. Output berupa grafik batang dan tabel yang menunjukkan bahwa fitur "Petal Length (cm)" dan "Petal Width (cm)" memiliki pengaruh terbesar terhadap hasil klasifikasi spesies bunga, sedangkan "Sepal Length (cm)" dan "Sepal Width (cm)" berpengaruh jauh lebih kecil

- Hyperparameter Tuning (Menentukan max_depth Terbaik)



Kode ini digunakan untuk mencari nilai maksimal kedalaman (max_depth) terbaik pada model decision tree dengan mencoba beberapa nilai dan mengevaluasi

akurasinya. Output dari proses ini menunjukkan bahwa nilai `max_depth` yang optimal adalah 3 dengan akurasi model tertinggi mencapai 96.67%, sehingga pemilihan hyperparameter dapat meningkatkan performa klasifikasi.

- Kesimpulan

Hasil praktikum mandiri klasifikasi dataset iris menggunakan algoritma Decision Tree menunjukkan bahwa model mampu memberikan akurasi sangat tinggi, yakni lebih dari 93% dan bahkan mencapai 96% setelah dilakukan tuning hyperparameter. Seluruh rangkaian proses mulai dari praproses data, encoding label, pemilihan fitur, splitting data, pelatihan, pengujian, hingga visualisasi dan tuning parameter berjalan dengan baik. Dua fitur yang paling berpengaruh dalam model adalah “Petal Length (cm)” dan “Petal Width (cm)”, sedangkan fitur sepal memberikan kontribusi lebih kecil. Model Decision Tree terbukti mudah diinterpretasikan melalui visualisasi pohon, sangat efektif untuk klasifikasi data sederhana seperti iris, serta performanya dapat dioptimalkan melalui pemilihan hyperparameter yang tepat.

Link Praktikum 05:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%205/Praktikum%20%26%20Tugas%20Mandiri%2005/notebooks/praktikum05.ipynb

Tugas Mandiri 05:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%205/Praktikum%20%26%20Tugas%20Mandiri%2005/notebooks/latihan05.ipynb